Introduction to Reinforcement Learning

Dr Aishwarya S., Assistant Professor of Mathematics, MIT, Manipal

Reinforcement Learning

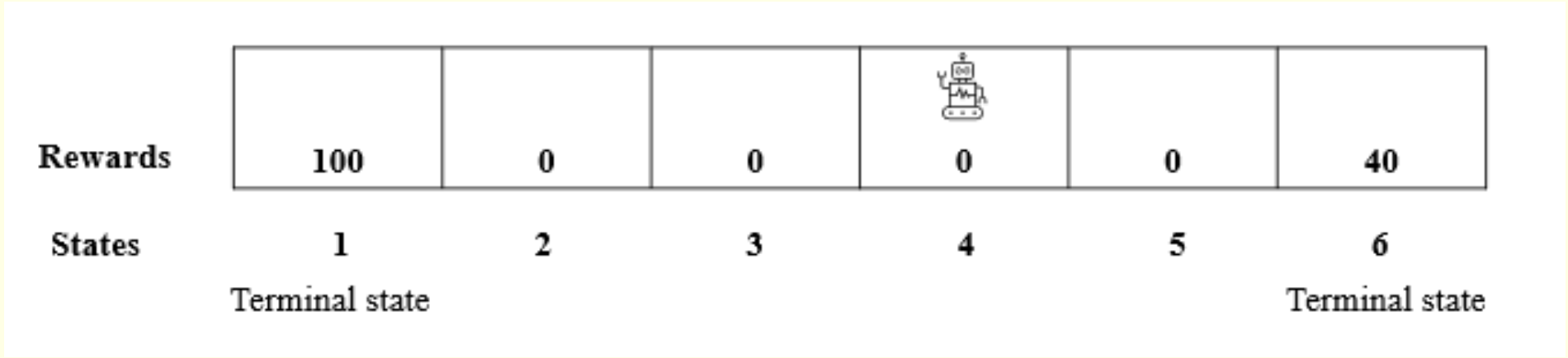
- Reinforcement Learning is a branch of machine learning where an agent learns to make sequential decisions by interacting with an environment.
- The agent observes the state, takes an action, and receives a reward as feedback.
- The goal of a reinforcement learning problem is to learn a strategy for choosing actions that maximizes cumulative reward.

- Unlike supervised learning, reinforcement learning does not rely on labeled examples. Instead, it learns through trial and error and delayed feedback.

- Reinforcement learning is used in areas such as robotics, game playing (Ex.: AlphaGo), recommendation systems, and autonomous driving.

- Consider a robot (agent) navigating in the following environment with six cells (called as **states**) namely, 1, 2, 3, 4, 5 and 6.
- Let 1 and 6 be terminal states.
- At any cell, the agent is allowed to make one of the two actions: ***left*** and ***right***.
- The agent must learn to navigate the environment with the help of rewards that it gets at every cell.

Rewards	**100**	**0**	**0**	 **0**	**0**	**40**
States	**1**	**2**	**3**	**4**	**5**	**6**
	Terminal state					Terminal state



Examples of sequences of states with associated rewards:

States: 4 3 2 1
Rewards: 0 0 0 100

States: 4 5 6
Rewards: 0 0 40

States: 4 5 4 3 2 1
Rewards: 0 0 0 0 0 100

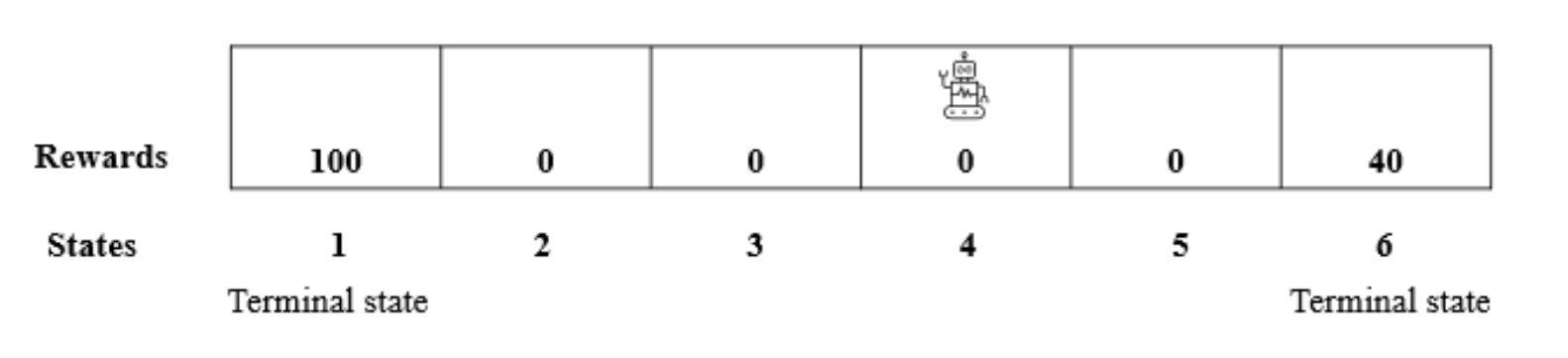
At every time step, the situation can be specified by the following 4-tuple:

$$(s, a, R(s), s')$$

where
s is the **state**,
a is the **action** taken by the agent in state s,
$R(s)$ is the reward at state s,
s' is the state reached after taking action a at state s.

Example: $(4, \leftarrow, 0,3)$

The concept of *Return*



Consider the following sequence of states:

States: 4 3 2 1
Rewards: 0 0 0 100

We compute **return** as follows:
Return $= 0 + (0.9)\ 0 + (0.9)^2\ 0 + (0.9)^3\ 100 = 72.9.$

Here, 0.9 is called the *discount factor*.

If $R_1, R_2, R_3, \ldots$ are the rewards at each state starting from time instance 1, then

Return $= R_1 + \gamma R_2 + \gamma^2 R_3 + \cdots$ (until a terminal state is reached).

Here γ is the discount factor. $(0 \leq \gamma \leq 1.)$

Find the returns for the following cases by taking discount factor $\gamma = 0.5$:

1. Starting at state 4 and moving ***left***.
2. Starting at state 3 and moving ***left***.
3. Starting at state 1.
4. Starting at state 4 and moving ***right***.
5. Starting at state 2 and moving ***right***.
6. Starting at state 6.

If the agent **always goes to *left***, then the returns at the states are as follows:

Returns	**100**	**50**	**25**	**12.5**	**6.25**	**40**
Rewards	**100**	**0**	**0**	**0**	**0**	**40**
States	**1**	**2**	**3**	**4**	**5**	**6**

If the agent **always goes to *right***, then the returns at the states are as follows:

Returns	**100**	**2.5**	**5**	**10**	**20**	**40**
Rewards	**100**	**0**	**0**	**0**	**0**	**40**
States	**1**	**2**	**3**	**4**	**5**	**6**

Note that for all the states except state 5,
Return by going to ***left*** $\geq$ Return by going to ***right*** .

So, it might be best for the agent to do something like the following:

If at state 2, go ***left***.
If at state 3, go ***left***.
If at state 4, go ***left***.
If at state 5, go ***right***.

In this case, the actions and the returns at the states are as follows:

Returns	**100**	**50**	**25**	**12.5**	**20**	**40**
		←	←	←	→	
Rewards	**100**	**0**	**0**	**0**	**0**	**40**
States	**1**	**2**	**3**	**4**	**5**	**6**

The discounted return at time t is defined as

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \cdots$$

where $R_t = R(s_t)$ is the reward for the state the agent is in at time t.

Therefore,

$$G_t = R_t + \gamma(R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots),$$

that is,

$$G_t = R_t + \gamma G_{t+1}.$$

Thus, the return from state s is equal to the sum of the reward in s and γ times the return from the next state.

Policy in a Reinforcement Learning Problem

A **policy** π is any function that takes a state as an input and outputs an action (the action to take while being at that state).

$$s \overset{\pi}{\longmapsto} a$$

The goal in an RL task is to come up with a strategy that tells what action to take at every state in order to maximize the cumulative reward (more precisely, the return). This strategy is called the "**optimal policy**".

Goal: Find a policy that maximizes the return G_t from every state.

The strategy:
If at state 2, go ***left***.
If at state 3, go ***left***.
If at state 4, go ***left***.
If at state 5, go ***right***.

corresponds to the following **policy**:

$\pi(2) = \pi(3) = \pi(4) = \boldsymbol{left}$;
$\pi(5) = \boldsymbol{right}$.

Note that the above is the best (optimal) policy in this example.
The optimal policy is denoted by π^*.

Formalism for RL Problems – Markov Decision Processes (MDPs)

A (**deterministic**) **Markov Decision Process** (MDP) consists of the following components:

- Set of states S.
- Set of actions A.
- State transition function $T: S \times A \to S$.
  ($T(s, a)$ is the next state when action a is taken in state s.)
- Reward function $R: S \to \mathbb{R}$.
  ($R(s)$ is the reward received for being in state s.)
- Discount factor $\gamma \in [0, 1)$.

Markov Property:

The next state depends only on the current state and action; not on the earlier history.

Value Functions of a Policy

For a policy π,

$$V^{\pi}(s) = \text{return } G_t \text{ starting in } s_t = s \text{ and following } \pi,$$
$$Q^{\pi}(s,a) = \text{return } G_t \text{ starting in } s_t = s, \text{ taking } a_t = a \text{ first, then following } \pi.$$

In words:

value of a state = reward for being in that state + γ (value of the state where the agent lands next).

At a terminal state, there is no next state; hence $V^{\pi}(s) = R(s)$.

Note that $V^{\pi}(s) = Q^{\pi}\big(s, \pi(s)\big)$.

V is the **state-value function** and Q the **action-value function**.

Bellman Equations

Using $G_t = R_t + \gamma G_{t+1}$,

$$V^{\pi}(s) = R(s) + \gamma\, V^{\pi}\big(T(s, \pi(s))\big)$$

and

$$Q^{\pi}(s,a) = R(s) + \gamma\, V^{\pi}\big(T(s,a)\big).$$

Optimal Value Functions

$$V^{*}(s) = \max_{\pi} V^{\pi}(s)$$

and

$$Q^{*}(s,a) = \max_{\pi} Q^{\pi}(s,a).$$

The optimal policy is derived from V^* or Q^*:

$$\pi^{*}(s) = \arg\max_{a} V^{*}(T(s,a)).$$

$$\pi^{*}(s) = \arg\max_{a} Q^{*}(s,a).$$

For finite discounted MDPs, a deterministic policy that is optimal from every state always exists.

Bellman Optimality Equations

$$V^{*}(s) = R(s) + \gamma \max_{a} V^{*}\big(T(s,a)\big)$$

$$Q^{*}(s,a) = R(s) + \gamma \max_{a'} Q^{*}\big(T(s,a), a'\big).$$

Terminal states: $V^{*}(s) = R(s)$.

Value Iteration Algorithm

1. Initialize $V(s) = 0$ for all s.
2. Repeat until convergence: {
   For every non-terminal s,

$$V_{\text{new}}(s) \;\leftarrow\; R(s) + \gamma \max_a V\big(T(s,a)\big).$$

   For every terminal s,

$$V_{\text{new}}(s) \leftarrow R(s).$$

   Set $V \leftarrow V_{\text{new}}$.
   }
3. Extract the policy: $\pi(s) = \arg\max_a V\big(T(s,a)\big)$. (This is the optimal policy π^*.)

Example

Consider the following 3×3 gridworld. Cell $(1,3)$ is the GOAL and Cell $(2,3)$ has a PIT.

$(1,1)$	$(1,2)$	**GOAL**
$(2,1)$	$(2,2)$	**PIT**
$(3,1)$	$(3,2)$	$(3,3)$

The actions are $\{Up(U), Down(D), Left(L), Right(R)\}$. Each action moves the agent one cell in that direction; moving into the wall leaves it where it is. The reward function is defined as follows: $R(\text{GOAL}) = +1$, $R(\text{PIT}) = -1$, and $R(s) = 0$ for every other state. GOAL and PIT are the terminal states. Discount $\gamma = 0.9$.

Each grid below shows V after the given iteration. The terminal cells show their values $R(s) = +1$ (GOAL) and $R(s) = -1$ (PIT) once they have been updated in Iteration 1.

Iteration 0

0	0	0
0	0	0
0	0	0

Iteration 1

0	0	1
0	0	-1
0	0	0

Iteration 2

0	0.9	1
0	0	-1
0	0	0

Iteration 3

0.81	0.9	1
0	0.81	-1
0	0	0

Iteration 4

0.81	0.9	1
0.729	0.81	-1
0	0.729	0

Iteration 5

0.81	0.9	1
0.729	0.81	-1
0.6561	0.729	0.6561

Iteration 6 (converged)

0.81	0.9	1
0.729	0.81	-1
0.6561	0.729	0.6561

Extracted optimal policy:

$\rightarrow$	$\rightarrow$	G
$\uparrow$ or $\rightarrow$	$\uparrow$	P
$\uparrow$ or $\rightarrow$	$\uparrow$	$\leftarrow$

Policy Iteration Algorithm

1. Start with any policy π (for example, arbitrary actions).
2. **Policy evaluation:** Compute V^{π} by repeating the update

$$V^{\pi}(s) \leftarrow R(s) + \gamma\, V^{\pi}\big(T(s, \pi(s))\big),$$

   until the values stop changing.
   (Note that there is no max here; the action at each state is determined by π.)
3. **Policy improvement:** Define a new policy that is greedy with respect to V^{π}:

$$\pi'(s) = \arg\max_{a} V^{\pi}\big(T(s, a)\big).$$

4. If $\pi' = \pi$, stop. (In this case, π is optimal.) Otherwise set $\pi \leftarrow \pi'$ and go to Step 2.

A note about convergence of the policy: The greedy step never makes the policy worse. Since there are finitely many deterministic policies, the loop terminates, and it terminates at an optimal policy, usually after a few iterations.

Example

Consider the 3×3 gridworld described earlier. Let us make the following choices:

- **Initial policy:** "always move left" ($\leftarrow$ in every non-terminal state).
  (Note that this is clearly a bad policy in the given gridworld.)
- **Evaluation** starts from $V = 0$ in every round (a warm start from the previous round's values also works and needs fewer iterations).
- **Tie-breaking in the improvement step:** Keep the current action unless some action is *strictly* better; if several actions are strictly better and tie, take the first in the order U, D, L, R. Keeping the current action on ties is what guarantees that the loop stops.

With these choices, the following is an illustration of the policy iteration algorithm implemented for the gridworld.

In each row, the left grid is the policy π_k being evaluated (red arrows are the actions changed by the previous improvement step) and the right grid is its value function V^{π_k} after evaluation has converged. The last column counts the evaluation iterations that changed the values.

Round k	Policy π_k	V^{π_k}	Number of iterations
1	← ← **G** ← ← **P** ← ← ←	0 0 1 0 0 −1 0 0 0	1
2	← → **G** ← ← **P** ← ← ←	0 0.9 1 0 0 −1 0 0 0	2
3	→ → **G** ← ↑ **P** ← ← ←	0.81 0.9 1 0 0.81 −1 0 0 0	3
4	→ → **G** ↑ ↑ **P** ← ↑ ←	0.81 0.9 1 0.729 0.81 −1 0 0.729 0.6561	5
5	→ → **G** ↑ ↑ **P** ↑ ↑ ←	0.81 0.9 1 0.729 0.81 −1 0.6561 0.729 0.6561	5

After round 5, the improvement step returns π_5 unchanged, so the algorithm stops: π_5 is optimal and $V^{\pi_5} = V^*$.

Note: Value iteration and policy iteration require knowledge of the transition model T and the reward function R. Therefore, they are **model-based** reinforcement learning algorithms. However, in many reinforcement learning problems, the agent does not know the transition model T or the reward function R beforehand. Instead, it observes experience tuples

$$(s_t, a_t, r_{t+1}, s_{t+1})$$

by interacting with the environment. This motivates **model-free** reinforcement learning methods, which learn value functions or policies directly from experience without requiring an explicit model of the environment. We will now look at **Q-learning**, which is a model-free reinforcement learning method.

Q-Learning

Q-learning proceeds over multiple episodes. During each episode, the agent interacts with the environment, observes transitions, and updates its action-value estimates $Q(s, a)$. Suppose the agent takes action a in state s, receives reward $R(s)$, and moves to the next state s'. The Q-learning update is

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[\underbrace{R(s) + \gamma \max_{a'} Q(s',a')}_{\text{target}} - \underbrace{Q(s,a)}_{\text{current estimate}}\Big],$$

where $\alpha \in (0, 1]$ is the learning rate and γ is the discount factor.

If s' is a terminal state, there are no future rewards to consider. Therefore, the future-value term is taken to be zero:

$$\max_{a'} Q(s', a') = 0.$$

The update then becomes

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[R(s) - Q(s,a)\big].$$

ε-Greedy Exploration. If the agent always chooses an action that maximizes $Q(s,a)$, it may never try actions that initially appear unfavourable but are actually better. Therefore, the following strategy is commonly used to choose an action in state s:

$$a = \begin{cases} \text{random action} & \text{with probability } \varepsilon, \\ \arg\max_a Q(s,a) & \text{with probability } 1-\varepsilon. \end{cases}$$

Typically, ε is initially set to a relatively large value and gradually decreased over the episodes. This encourages exploration early in training and exploitation of the learned action values later.

Q-Learning Algorithm

1. Initialize $Q(s,a) = 0$ for all s, a.
2. For each episode: {
   Start in an initial state s.
   Repeat until the episode ends: {
   Choose action a by ε-greedy exploration, observe state s', and apply the update

$$Q(s,a) \;\leftarrow\; Q(s,a) + \alpha\Big[R(s) + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big].$$

   Set $s \leftarrow s'$.
   } }
3. After many episodes, compute $\pi(s) = \arg\max_a Q(s,a)$.

Convergence: Q-learning converges to Q^* if every state-action pair keeps being visited.

Example computations ($\alpha = 1$, $\gamma = 0.9$, all $Q = 0$ initially)

The moves below are chosen by hand to illustrate how information propagates. A transition into a terminal state uses the reward of that state (+1 for GOAL) in place of $\max_{a'} Q(s',a')$.

#	Transition $(s, a \to s')$	Computation	New Q
1	$(1,2)$, $R \to$ GOAL	$0 + 1(0 + 0.9(1) - 0)$	$Q((1,2),R) = 0.9$
2	$(1,1)$, $R \to (1,2)$	$0 + 1(0 + 0.9(0.9) - 0)$	$Q((1,1),R) = 0.81$

Stochastic Markov Decision Processes

In most RL problems, actions do not always do what the agent intends. Such RL problems are described as **stochastic MDPs**.

In a stochastic MDP, the resulting state after taking an action a at state s is given by a probability distribution over different states, instead of the (deterministic) state transition function $T: S \times A \rightarrow S$.

A **stochastic MDP** consists of the following components:

- Set of states S.
- Set of actions A.
- Probabilities $P(s'|s,a)$ of moving to state s' when action a is taken at state s.
- Reward function $R: S \to \mathbb{R}$.
  ($R(s)$ is the reward received for being in state s.)
- Discount factor $\gamma \in [0,1)$.

Note that a deterministic MDP is a special case of stochastic MDP where $P(s'|s,a) = 1$ for $s' = T(s,a)$, and 0 for all $s' \neq T(s,a)$.

In a stochastic MDP, the return G_t is a random variable.
Hence the value functions are **expected returns**:

$$V^{\pi}(s) = E_{\pi}[G_t \mid s_t = s]$$

$$Q^{\pi}(s,a) = E_{\pi}[G_t \mid s_t = s, a_t = a]$$

The goal of a stochastic RL task is to choose a policy $\pi: S \to A$ which gives the best action a to take from state s so as to **maximize the expected return**.

Bellman Equations for a Stochastic MDP

$$V^{\pi}(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi(s))\, V^{\pi}(s').$$

$$Q^{\pi}(s,a) = R(s) + \gamma \sum_{s'} P(s' \mid s, a)\, V^{\pi}(s').$$

$$V^{*}(s) = R(s) + \gamma \max_{a} \sum_{s'} P(s' \mid s, a)\, V^{*}(s').$$

$$Q^{*}(s,a) = R(s) + \gamma \sum_{s'} P(s' \mid s, a) \max_{a'} Q^{*}(s', a').$$

For a terminal state s, $V(s) = R(s)$ (same as in a deterministic MDP).

The optimal policy is given by

$$\pi^*(s) = \arg\max_a Q^*(s, a).$$

Note:

Value iteration, policy iteration, and Q-learning for stochastic MDPs use update rules based on the corresponding Bellman equations.

While value iteration and policy iteration compute expected returns using the transition model, Q-learning learns from sampled transitions.